dsdfit: Estimating Turbulent Broadening from Droplet Size Histograms

A guide to the method, the code, and what to try

Version 0.1 October 2026

Contents

1 What this is for **1**

2 The physics in plain words **2**

3 What the method does **4**

4 Using the code **5**
4.1 Installing 5
4.2 The functions 6
4.3 The histogram format 7
4.4 Examples 7

5 The main task: C-ODT **8**

6 Suggested warm-ups **9**

A Glossary **10**

B How the fit works **10**

C Troubleshooting **11**

1 What this is for

Clouds are made of water droplets of many different sizes. How widely those sizes spread out matters: a broad spread of sizes is one of the first steps toward rain. One reason the sizes spread out is turbulence. Different droplets experience different amounts of humidity as they are tossed around, so they grow at different rates.

A model called the *Fokker–Planck equation* describes this. It predicts the distribution of droplet sizes from a few physical quantities. Two of them are hard to measure directly:

- $\bar{s}$, the **mean supersaturation**: on average, how much extra water vapor the air holds;
- D_x, the **broadening**: how strongly fluctuations in that humidity spread droplet sizes apart.

dsdfit estimates D_x and $\bar{s}$, with error bars, from a histogram of droplet sizes. It can also make synthetic histograms from the model, so you can test the method before trusting it.

Your main task. Simulations such as C-ODT (the Cloudy One-Dimensional Turbulence model) produce droplet sizes *and* record the humidity fluctuations that caused them. The model predicts

$$D_x \approx a^2 \sigma_s^2 \, \tau, \tag{1}$$

where σ_s measures how big the humidity fluctuations are and τ how long they last (Section 2). The task is to fit C-ODT size histograms with dsdfit and check whether the fitted D_x agrees with $a^2\sigma_s^2\tau$ computed from C-ODT's own fluctuations (Section 5). If it does, that is strong evidence that the method works on realistic data. If it does not, finding out why is just as valuable.

2 The physics in plain words

Supersaturation. Air at a given temperature can hold only so much water vapor; at that point it is *saturated*. The **supersaturation** s is how much extra vapor the air holds beyond saturation, as a fraction: $s = 0.001$ means 0.1% more vapor than saturation. When $s > 0$, water condenses onto droplets and they grow; when $s < 0$, they evaporate. In clouds, s is tiny, usually well under 1%. Throughout dsdfit, s and $\bar{s}$ are written as fractions.

Droplets form on salt particles. Droplets do not form from nothing: each one starts on a tiny particle, here a grain of salt about 0.1 µm across. Dissolved salt helps a small droplet take up water, while the curvature of a very small droplet works against it. A droplet has to grow past a *critical size* before it can keep growing; this is called *activation*. The size of the dry salt particle matters for this step.

Growth. Once activated, a droplet of radius r grows as

$$\frac{d(r^2)}{dt} = a\,\big(s - s_{\mathrm{eq}}(r)\big), \tag{2}$$

where a is a *growth constant* set by the temperature and pressure, and $s_{\mathrm{eq}}(r)$ is a small correction from salt and curvature that only matters for the smallest droplets. Notice that it is the *square* of the radius that grows at a steady rate. That is why the model works with $x = r^2$.

Small droplets grow a little more slowly (optional). Right at a droplet's surface, vapor and heat are exchanged molecule by molecule, and not every vapor molecule that hits the surface sticks. This slows small droplets by a factor $r/(r+\ell)$. The *kinetic length* ℓ is set by two *accommodation coefficients*: α_c, the fraction of vapor molecules that stick, and α_T, how well heat is passed to the air. By default dsdfit leaves this out ($\ell = 0$). To include it, give the coefficients: `Conditions(alpha_c=0.04, alpha_T=1.0)` uses C-ODT's values, which give $\ell \approx 1.5\,\mu\mathrm{m}$ at 288 K. Equation (2) then becomes

$$\frac{d(r+\ell)^2}{dt} = a\,\big(s - s_{\mathrm{eq}}(r)\big),$$

so the model works with $y = (r+\ell)^2$ instead of r^2, and nothing else changes. In particular, $D_x \approx a^2\sigma_s^2\tau$ still holds, now as the spreading of y.

Turbulence spreads sizes apart. In turbulent air the supersaturation fluctuates around its mean $\bar{s}$. The size of the fluctuations is measured by their standard deviation σ_s, and how long a typical fluctuation lasts by a time scale τ. Each droplet experiences its own random sequence of fluctuations, so identical droplets end up with different sizes. In the model this spreading acts like diffusion in x, with strength D_x; if the fluctuations are brief compared with the time a droplet takes to grow, then $D_x \approx a^2\sigma_s^2\tau$, which is equation (1).

Falling out. Bigger droplets fall faster: the falling speed is proportional to r^2. In a well-mixed chamber of height H, droplets of radius r are removed at a rate kr^2, where the *fallout constant* k is set by the falling speed and the chamber height.

Faster fallout (optional). If droplets are not spread evenly in height, they are removed at a different rate. Falling droplets tend to crowd toward the floor, where they leave, so they are removed faster than kr^2, and more so for big droplets, which fall fastest. dsdfit lets you multiply the rate by a *fallout factor* $f(r)$, so that droplets are removed at rate $kr^2 f(r)$. Give a number for the same change at every size, `Conditions(fallout_factor=1.3)`, or a function of the radius in µm, `Conditions(fallout_factor=lambda r:  1 + 0.005 * r**2)`. The default is $f = 1$.

Steady state. New particles are injected continuously, grow, spread apart, and eventually fall out. When these processes balance, the size distribution stops changing. The model predicts this *steady* distribution (Figure 1).

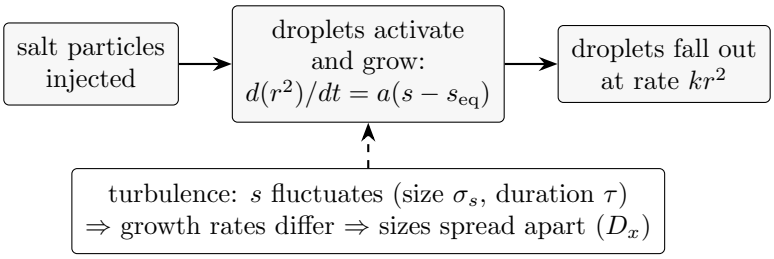


Figure 1: The life of a droplet in the model. In steady state, injection, growth, spreading and fallout balance, and the size distribution no longer changes.

The model. Writing $N(x)$ for the number of droplets per unit $x = r^2$, the steady Fokker–Planck equation is

$$\frac{d}{dx}\Big[\underbrace{a\Big(\bar{s} - \frac{A_k}{r} + \frac{B_k}{r^3}\Big)N}_{\text{growth}} - \underbrace{D_x\,\frac{dN}{dx}}_{\text{spreading}}\Big] + \underbrace{k\,x\,N}_{\text{falling out}} = 0. \tag{3}$$

The bracket is the rate at which droplets move toward larger sizes. Droplets are injected at the smallest size and none go past the largest. With the kinetic correction, x is replaced by $y = (r+\ell)^2$; the fallout term stays kr^2N. With a fallout factor, it becomes $kr^2 f(r)N$. You will not need to solve this yourself; dsdfit does it.

The parameters. Table 1 lists everything that enters. The first two are estimated; the next two are what C-ODT can provide for the comparison; the rest you set to match your data, and dsdfit computes the model constants from them.

Symbol	Meaning	Units	Example	In dsdfit
$\bar{s}$	mean supersaturation	fraction	0.0009	**estimated** (`s_bar`)
D_x	how strongly fluctuations spread sizes	$\mu m^4/s$	0.3	**estimated** (`D_x`)
σ_s	size of the supersaturation fluctuations	fraction	0.002	from C-ODT, for comparison
τ	how long a fluctuation lasts	s	2	from C-ODT, for comparison
T	air temperature	K	288	`Conditions(T=...)`
p	air pressure	Pa	1.0×10^5	`Conditions(p=...)`
d_{dry}	diameter of the dry salt particle	µm	0.13	`Conditions(d_dry=...)`
H	chamber height	m	1	`Conditions(H=...)`
α_c, α_T	accommodation coefficients (optional)	fraction	0.04, 1	`Conditions(alpha_c=...)`
$f(r)$	fallout factor (optional)		1	`Conditions(fallout_factor=...)`
a	growth constant	$\mu m^2/s$	206	computed (`cond.a`)
A_k, B_k	curvature and salt constants	µm, μm^3		computed
k	fallout constant	$s^{-1}\mu m^{-2}$	1.2×10^{-4}	computed (`cond.k`)
ℓ	kinetic length	µm	0	computed (`cond.ell`)

Table 1: The parameters. Example values are the defaults used in this guide. The accommodation coefficients are not set by default ($\ell = 0$); 0.04 and 1 are C-ODT's values.

What a histogram can and cannot tell you.

- It tells you the **shape** of the distribution, which depends on D_x and $\bar{s}$ (Figure 2). It does not depend on how many particles were injected.
- It **cannot separate σ_s from τ**: they only enter through D_x. That is why the comparison uses their combination $a^2\sigma_s^2\tau$.
- It relies on the growth constant a and fallout constant k being right. If the conditions you give are wrong, the estimates will be off. The fit check (Section 3) can sometimes tell you, but not for the fallout rate: the shape depends only on how fast droplets grow and spread *compared with* how fast they fall out, so a fallout rate that is wrong by the same factor at every size gives a perfect-looking fit with wrong estimates.

3 What the method does

The data. A histogram records how many droplets fell in each size bin. It carries two separate pieces of information: the *shape* (what fraction of droplets is in each bin) and the *number of droplets* n. The model predicts the shape. The number of droplets tells you how noisy the histogram is: a histogram of 100 droplets is ragged, one of 100,000 is smooth.

The fit. `fit` tries values of D_x and $\bar{s}$, computes the predicted fractions, and measures the mismatch with the observed counts with a number J (the *loss*). It searches for the pair that makes J smallest. That pair is the estimate. How the mismatch is measured and how the search works are in Appendix B; you do not need them to use the code.

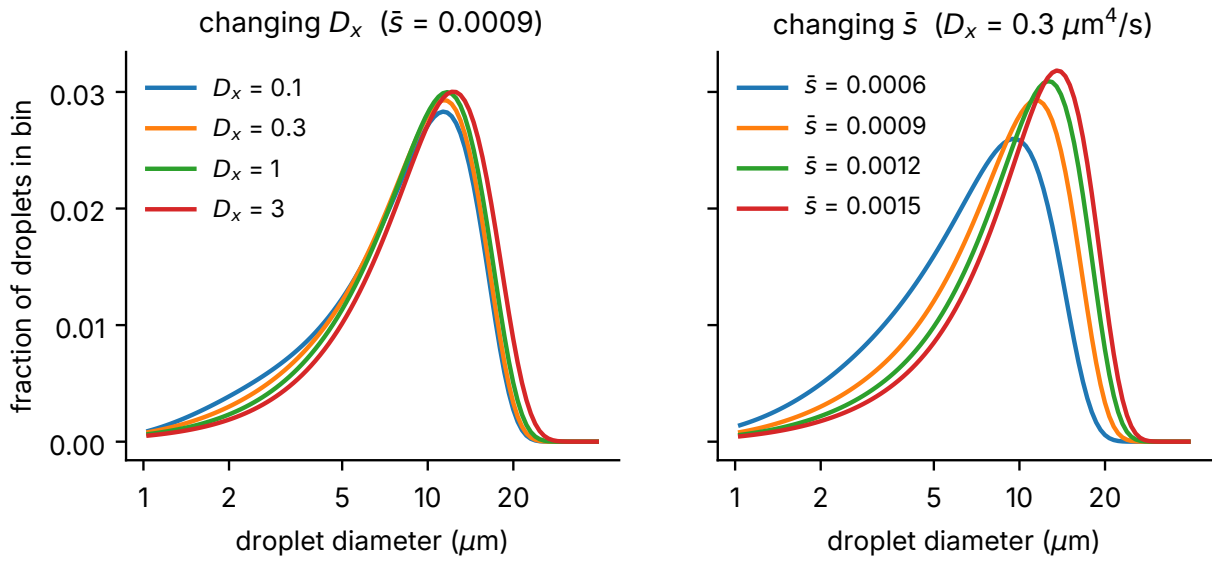


Figure 2: The model's predicted fraction of droplets in each bin. Raising D_x (left) or $\bar{s}$ (right) both move droplets toward larger sizes, in somewhat different ways. Because the effects are similar, the two estimates end up correlated.

Error bars. Each estimate comes with an error bar of one standard deviation. If you repeated the experiment many times, the estimate would land within one error bar of the true value about 2 times in 3, and within two error bars about 19 times in 20. Error bars shrink as the number of droplets grows. `fit` also reports the *correlation* between the two estimates. It is usually strongly negative: a histogram that slightly favors a larger D_x also favors a smaller $\bar{s}$, because the two have similar effects (Figure 2).

The fit check. Even with the true parameters, a histogram never matches the model exactly, because of counting noise. dsdfit knows roughly how large J should be when the model is right (`J_expected`) and compares it with the value at the best fit (`J_min`). If `J_min` is much larger, the model cannot describe the data: the conditions may be wrong, or the data may break one of the model's assumptions.

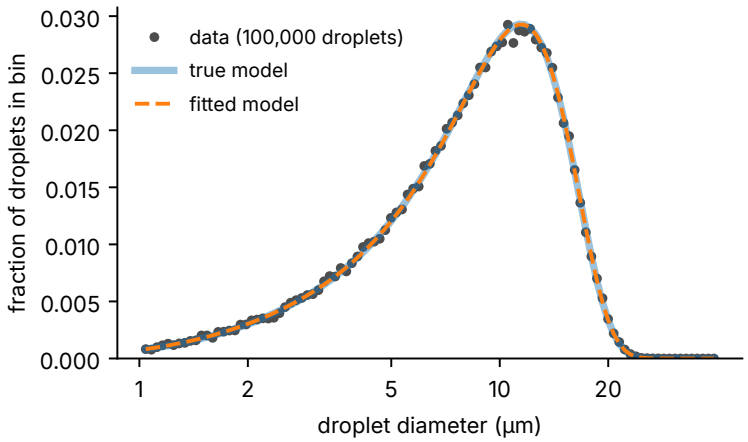
Assumptions. The method assumes (1) the distribution is in steady state; (2) each droplet is counted once and independently of the others; (3) every droplet in the histogram's size range is counted equally well; and (4) the conditions (temperature, pressure, particle size, chamber height) are right.

4 Using the code

4.1 Installing

```
git clone https://github.com/dawsoncollins99/dsdfit.git
cd dsdfit
pip install -e ".[notebook]"
```

Then open `examples/example.ipynb`, which walks through everything in this section.




```
D_x   = 0.278 +/- 0.02 um^4/s   (7.3%)
s_bar = 0.00090578 +/- 5.7e-06      (0.09058% +/- 0.00057 percentage points)
correlation between the two estimates: -0.84
fit check: J_min = 77.4, expected about 90 +/- 13 -> consistent with the model
(100000 droplets, 4 iterations)
```

Figure 3: One synthetic histogram of 10^5 droplets (true $D_x = 0.3$, $\bar{s} = 0.0009$), the fitted model, and the summary printed by `print(result)`.

4.2 The functions

Function	What it does
`Conditions(T, p, d_dry, H, alpha_c, alpha_T, fallout_factor)`	physical conditions; computes a, A_k, B_k, k, ℓ (`print(cond)` shows them)
`welas_bins(cutoff_diameter=1.0)`	bin edges of the WELAS instrument: 103 bins from 1 to 40 µm
`Histogram(edges, counts)`	droplet counts in size bins (Section 4.3)
`load_histogram(path)`, `save_histogram(h, path)`	read and write histograms as CSV files
`model_fractions(D_x, s_bar, cond, edges)`	the model's predicted fraction of droplets in each bin
`simulate(D_x, s_bar, cond, edges, n, seed)`	a synthetic histogram of `n` droplets
`model_evolution(D_x, s_bar, cond, edges, times)`	droplet numbers in each bin over time, after a pulse of aerosol or from an empty chamber
`fit(hist, cond)`	estimates of D_x and $\bar{s}$ with error bars, as a `FitResult`
`predicted_D_x(sigma_s, tau, cond)`	$a^2\sigma_s^2\tau$, to compare with a fitted D_x
`plot_fit(hist, result)`	plot a histogram with the true and fitted model

A `FitResult` has the attributes `D_x`, `D_x_err`, `s_bar`, `s_bar_err`, `correlation`, `J_min`, `J_expected`, `fit_ok`, `n`, `converged` and `fractions` (the fitted model's fraction in each bin).

Conventions. Sizes you type are droplet **diameters in micrometers**. D_x is in $\mu m^4/s$. $\bar{s}$ and σ_s are **fractions** (0.0009 means 0.09%). Times are in seconds.

4.3 The histogram format

Everything `fit` needs from the data is a histogram: bin edges and counts. In Python, `Histogram(edges, counts)`; on disk, a CSV file with one row per bin:

```
# comment lines start with #
lower_diameter_um,upper_diameter_um,count
1.0233,1.06038,88
1.06038,1.0988,75
...
```

The rules:

- **Edges are diameters in µm**, strictly increasing. Bins must be contiguous: each upper edge is the next bin's lower edge. Bins do not have to be the WELAS bins.
- **Counts are whole numbers of droplets**, one per bin. A normalized distribution, a concentration, or a probability density is *not* a valid set of counts. `Histogram` refuses non-whole numbers.
- **Each droplet is counted once, independently.** The error bars depend on this. If you add up many snapshots of a simulation that contain the same droplets, each droplet is counted many times: the estimates are still fine, but the error bars will be too small.
- **Use plenty of droplets.** dsdfit warns below 10,000; with fewer, error bars are less reliable.

4.4 Examples

Simulate and fit. This produces Figure 3.

```
from dsdfit import Conditions, welas_bins, simulate, fit, plot_fit

cond  = Conditions()            # T = 288 K, p = 1e5 Pa, d_dry = 0.13 um, H = 1 m
edges = welas_bins()            # 103 bins from 1 to 40 um
hist  = simulate(D_x=0.3, s_bar=0.0009, cond=cond, edges=edges, n=100_000, seed=1)
result = fit(hist, cond)
print(result)
plot_fit(hist, result)
```

Does it recover the truth? Repeat with different random seeds and compare the estimates with the true values (Figure 4). The notebook contains the plotting code.

```
results = [fit(simulate(0.3, 0.0009, cond, edges, n=100_000, seed=s), cond)
           for s in range(20)]
```

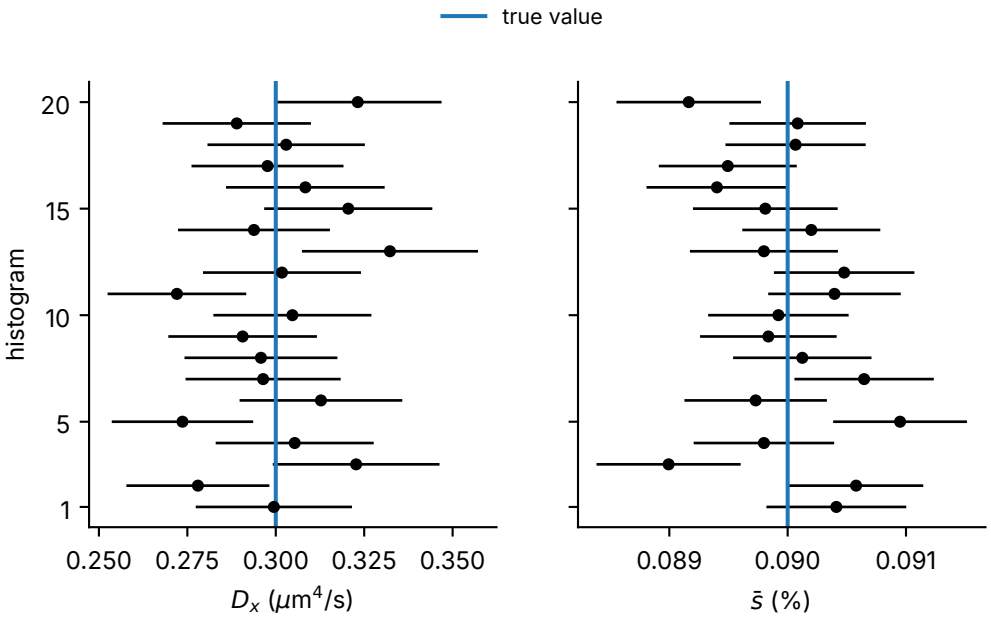


Figure 4: Twenty synthetic histograms of 10^5 droplets each, fitted with the same conditions that made them. The true value lies within one error bar for 16 of the 20 D_x estimates and 14 of the 20 $\bar{s}$ estimates, and within two error bars for all of them, as expected.

Fitting with different conditions. The conditions used to *make* the data and the conditions the fit *assumes* are separate arguments, so you can ask what happens when the fit assumes something slightly wrong:

```
hist = simulate(0.3, 0.0009, Conditions(H=1.0), edges, n=100_000, seed=1)
result = fit(hist, Conditions(H=1.1))       # the fit assumes a chamber 10% taller
```

Data from a file. This is how other data, such as a C-ODT histogram, come in:

```
from dsdfit import load_histogram, predicted_D_x
hist = load_histogram("data/example_histogram.csv")
result = fit(hist, cond)
D_pred = predicted_D_x(sigma_s=0.0019, tau=2.0, cond=cond)
print((result.D_x - D_pred) / result.D_x_err, "error bars apart")
```

5 The main task: C-ODT

The goal is to find out whether the fitted D_x agrees with $a^2\sigma_s^2\tau$ computed from C-ODT's supersaturation record. Here is one way to go about it.

1. **Pick a steady period.** After a simulation starts, the size distribution takes a while to settle. Use only output after it has stopped changing (compare histograms from early and late in the period).

2. **Build the histogram.** Count droplets in diameter bins and write the CSV file (Section 4.3). You can use the WELAS bins, to mimic the instrument, or your own bins over all sizes. Make

sure each count is a number of droplets, and think about whether the same droplets appear in several of the snapshots you combine.

3. **Set the conditions.** Use the run's temperature, pressure, dry particle size and domain height. C-ODT includes the kinetic correction with $\alpha_c = 0.04$ and $\alpha_T = 1$, so pass those too. Check how C-ODT removes droplets: dsdfit assumes they fall out at rate kr^2, with k set by the chamber height. C-ODT removes a droplet when it reaches the floor, so you can count removals by size in its output, divide by kr^2 times the number of droplets of that size, and pass the result as `fallout_factor`.

4. **Fit.** Look at the estimates, the error bars, and the fit check. Plot the histogram with the fitted model.

5. **Compute σ_s and τ** from the supersaturation experienced by droplets over the same period. With $s'(t) = s(t) - \bar{s}$ the fluctuation,

$$\sigma_s^2 = \text{average of } s'^2, \qquad \rho(t) = \frac{\text{average of } s'(t_0)\, s'(t_0 + t)}{\sigma_s^2}, \qquad \tau = \int_0^\infty \rho(t)\, dt.$$

   $\rho(t)$ is the autocorrelation: how similar the supersaturation is to itself a time t later. It starts at 1 and decays; τ is the area under it. In practice, integrate until ρ first drops to zero.

6. **Compare.** Compute `predicted_D_x(sigma_s, tau, cond)` and compare it with the fitted D_x, in units of the error bar. Also compare the fitted $\bar{s}$ with C-ODT's mean supersaturation.

If they disagree. Some possible reasons, each worth checking:

- the conditions do not match the run (especially anything affecting growth or fallout);
- the histogram includes particles that have not activated, which the model's spreading term cannot describe (with 0.13 µm salt particles, activation happens near 2 µm diameter; try `welas_bins(3.0)`);
- the period is not steady, or the counts are not independent (error bars too small);
- C-ODT injects or removes droplets differently from the model;
- the fluctuations last too long for $D_x \approx a^2 \sigma_s^2 \tau$ to hold (it needs τ short compared with the time a droplet takes to grow);
- σ_s and τ were computed from supersaturation at fixed points in space rather than as experienced by the droplets.

6 Suggested warm-ups

Before working with C-ODT data, a few experiments with synthetic data will help you know what to expect. For each, predict the answer before running it.

- **Recovery at your run's conditions.** Repeat the recovery experiment of Figure 4 with the C-ODT run's conditions and plausible values of D_x and $\bar{s}$.
- **More droplets.** How do the error bars change as n goes from 10^3 to 10^6? Plot them on log–log axes.

- **Wrong conditions.** Simulate with one set of conditions and fit with another (for example, a chamber height 10% off). How far off are the estimates, measured in error bars? Does the fit check notice?
- **The detection limit.** Compare `welas_bins(1.0)`, `welas_bins(2.0)` and bins covering all sizes. How much do the error bars change?

A Glossary

Activation a droplet growing past its critical size, after which it keeps growing.

Autocorrelation how similar a fluctuating quantity is to itself a time t later.

Correlation (of estimates) how much an error in one estimate tends to come with an error in the other; between -1 and 1.

Error bar here, one standard deviation of the estimate.

Fallout droplets settling out of the chamber under gravity.

Fokker–Planck equation an equation for how a distribution changes under drift (here, growth), diffusion (here, spreading by turbulence) and removal.

Histogram counts of droplets in size bins.

Steady state a situation where the distribution no longer changes in time.

Supersaturation how much water vapor the air holds beyond saturation, as a fraction.

B How the fit works

This appendix is for anyone curious about what happens inside `fit`. Notation: the histogram has counts c_k in bins $k = 1, \dots, K$, with $n = \sum_k c_k$ droplets; $p_k(\theta)$ is the model's fraction in bin k; and $\theta = (\ln D_x, \bar{s})$.

Solving the model. Equation (3) is solved with finite volumes on a fine, uniform grid in $y = (r + \ell)^2$ (that is, $x = r^2$ when $\ell = 0$), using Scharfetter–Gummel fluxes, which stay accurate whether growth or spreading dominates. The result is one tridiagonal linear system. The solution is summed over each bin and normalized to give p_k.

The loss. If each droplet lands in bin k with probability p_k, independently of the others, the probability of the observed counts is proportional to $\prod_k p_k^{c_k}$. Maximizing it is the same as minimizing

$$J(\theta) = 2 \sum_k c_k \ln \frac{c_k}{n\, p_k(\theta)}, \tag{4}$$

which is zero only if the model matches the histogram exactly. For large counts, $J \approx \sum_k (c_k - np_k)^2/(np_k)$: a least-squares mismatch in which each bin is weighted by its expected noise.

The search (Gauss–Newton). Starting from the best of a coarse grid of precomputed shapes, each step solves a 2×2 linear system,

$$n\,\mathcal{I}\,\Delta\theta = \sum_k \frac{c_k - n p_k}{p_k}\,\frac{\partial p_k}{\partial \theta}, \qquad \mathcal{I}_{ij} = \sum_k \frac{1}{p_k}\,\frac{\partial p_k}{\partial \theta_i}\,\frac{\partial p_k}{\partial \theta_j}, \tag{5}$$

and moves along $\Delta\theta$: it tries the full step and the bottom of a parabola fitted to J along the step, keeps whichever gives the lower J, and halves the step if J still does not go down (a very long step is capped first). $\Delta\theta$ is the step that would fit the current mismatch exactly if the model were linear near θ. It usually converges in a handful of steps. The derivatives $\partial p_k/\partial\theta$ come from two more solves of the same tridiagonal system (the *tangent-linear* method).

Error bars. The matrix $\mathcal{I}$ is the *Fisher information*: how much one droplet tells you about θ. With n droplets, the covariance of the estimates is

$$\Sigma = (n\,\mathcal{I})^{-1}, \tag{6}$$

so the error bar on $\ln D_x$ is $\sqrt{\Sigma_{11}}$ (a relative error on D_x), the error bar on $\bar{s}$ is $\sqrt{\Sigma_{22}}$, and their correlation is $\Sigma_{12}/\sqrt{\Sigma_{11}\Sigma_{22}}$. These formulas assume many droplets and independent counts.

The fit check. When the model is right, $J_{\min}$ behaves roughly like a sum of one squared, standardized error per bin, so its typical value is about the number of well-populated bins minus 3 (one for the fixed total and one for each estimated parameter), with a spread of about $\sqrt{2\,J_{\text{expected}}}$. `fit_ok` is true when $J_{\min}$ is within three spreads of that value.

C Troubleshooting

"counts must be whole numbers of droplets" The data are a normalized distribution or a concentration. Convert them to the number of independent droplets they represent.

"Histogram has only ... droplets" A warning, not an error: with fewer than about 10,000 droplets, the error bars are less reliable.

`fit_ok` is False The model does not describe the data well. Check the conditions, whether the data are in steady state, and whether the counts are independent.

`converged` is False Try a starting guess close to where you expect the answer: `fit(hist, cond, start=(D_x, s_bar))`.

Very large error bars, correlation near ± 1 The histogram cannot tell D_x and $\bar{s}$ apart in this situation; more droplets, or bins covering more sizes, may help.

"The model puts (almost) no droplets in the bins" The bins lie where the model predicts no droplets (for example, all far beyond the largest droplets), or the parameters are extreme.

"needs too fine a grid" The bins extend to very large sizes. Drop empty bins far beyond the largest droplets.